

# **Лабораторная работа № 6. Решение моделей в непрерывном и дискретном времени**

Абакумова Олеся Максимовна, НФИбд-02-22

# Содержание

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                                  | <b>6</b>  |
| <b>2</b> | <b>Задание</b>                                      | <b>7</b>  |
| <b>3</b> | <b>Выполнение лабораторной работы</b>               | <b>8</b>  |
| 3.1      | Предварительные сведения . . . . .                  | 8         |
| 3.1.1    | Решение обыкновенных дифференциальных уравнений . . | 8         |
| 3.1.2    | Модель Лотки–Вольтерры . . . . .                    | 14        |
| 3.2      | Задания для самостоятельного выполнения . . . . .   | 16        |
| <b>4</b> | <b>Выводы</b>                                       | <b>37</b> |

# Список иллюстраций

|      |                                                                               |    |
|------|-------------------------------------------------------------------------------|----|
| 3.1  | Установка и подключение пакета <code>diffrentialEquations.jl</code> . . . . . | 9  |
| 3.2  | Численное решение . . . . .                                                   | 10 |
| 3.3  | Модель экспоненциального роста . . . . .                                      | 10 |
| 3.4  | Задание точности решения . . . . .                                            | 11 |
| 3.5  | Модель экспоненциального роста . . . . .                                      | 11 |
| 3.6  | Численное решение . . . . .                                                   | 13 |
| 3.7  | Аттрактор Лоренца . . . . .                                                   | 13 |
| 3.8  | Аттрактор Лоренца (интерполяция отключена) . . . . .                          | 14 |
| 3.9  | Подключение пакета <code>ParameterizedFunctions</code> . . . . .              | 15 |
| 3.10 | Численное решение . . . . .                                                   | 15 |
| 3.11 | Модель Лотки–Вольтерры: динамика изменения численности популяций . . . . .    | 16 |
| 3.12 | Модель Лотки–Вольтерры: фазовый портрет . . . . .                             | 16 |
| 3.13 | Модель Мальтуса . . . . .                                                     | 17 |
| 3.14 | График модели Мальтуса . . . . .                                              | 18 |
| 3.15 | Анимация модели Мальтуса . . . . .                                            | 18 |
| 3.16 | Логистическая модель роста популяции . . . . .                                | 19 |
| 3.17 | График логистической модели роста популяции . . . . .                         | 20 |
| 3.18 | Анимация графика логистической модели роста популяции . . . . .               | 20 |
| 3.19 | Модель эпидемии SIR . . . . .                                                 | 22 |
| 3.20 | График модели эпидемии SIR . . . . .                                          | 22 |
| 3.21 | Отдельные графики модели эпидемии SIR . . . . .                               | 23 |
| 3.22 | Анимация модели эпидемии SIR . . . . .                                        | 23 |
| 3.23 | Модель SEIR . . . . .                                                         | 25 |
| 3.24 | График модели SEIR . . . . .                                                  | 25 |
| 3.25 | Сравнение SIR и SEIR . . . . .                                                | 26 |
| 3.26 | Графики сравнения SIR и SEIR . . . . .                                        | 27 |
| 3.27 | Анализ результатов . . . . .                                                  | 27 |
| 3.28 | Модель Лотки-Вольтерры . . . . .                                              | 28 |
| 3.29 | Фазовый портрет модели Лотки-Вольтерры . . . . .                              | 29 |
| 3.30 | Фазовый портрет с точками равновесия . . . . .                                | 29 |
| 3.31 | Проверка сходимости к точке равновесия . . . . .                              | 30 |
| 3.32 | Модель конкурентных отношений . . . . .                                       | 31 |
| 3.33 | График временных зависимостей и фазовый портрет . . . . .                     | 32 |
| 3.34 | Анимация модели конкурентных отношений . . . . .                              | 32 |
| 3.35 | Консервативный гармонический осциллятор . . . . .                             | 33 |

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| 3.36 График консервативного гармонического осциллятора и фазовый<br>портрет . . . . . | 34 |
| 3.37 Анимация фазового портрета . . . . .                                             | 34 |
| 3.38 Свободные колебания гармонического осциллятора . . . . .                         | 35 |
| 3.39 График затухающего гармонического осциллятора и его фазовый<br>портрет . . . . . | 36 |
| 3.40 Анимация фазового портрета . . . . .                                             | 36 |

## **Список таблиц**

# 1 Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

## **2 Задание**

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

## 3 Выполнение лабораторной работы

### 3.1 Предварительные сведения

#### 3.1.1 Решение обыкновенных дифференциальных уравнений

Напомним, что обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной  $u$ :

$$u'(t) = f(u(t), p, t)$$

,

где

$$f(u(t), p, t)$$

— нелинейная модель (функция) изменения  $u(t)$  с заданным начальным значением

$$u(t_0) = u_0$$

,  $p$  — параметры модели,  $t$  — время. Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет `differentialEquations.jl`.

##### 3.1.1.1 Модель экспоненциального роста

Рассмотрим пример использования этого пакета для решения уравнения модели экспоненциального роста, описываемую уравнением



$$u'(t) = au(t), \quad u(0) = u_0$$

,

где  $a$  — коэффициент роста. Предположим, что заданы следующие начальные данные  $a = 0,98$ ,  $u(0) = 1,0$ ,  $t \in [0; 1,0]$ .

Аналитическое решение модели имеет вид:

$$u(t) = u_0 \exp(at)$$

Численное решение в Julia будет иметь следующий вид (рис. 3.1):

```
In [1]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")
```

|           |                               |          |
|-----------|-------------------------------|----------|
| Installed | SciMLBase                     | v2.127.0 |
| Installed | SimpleUnPack                  | v1.1.0   |
| Installed | OrdinaryDiffEqFIRK            | v1.16.0  |
| Installed | NonlinearSolveQuasiNewton     | v1.11.0  |
| Installed | BoundaryValueDiffEqMIRKN      | v1.8.0   |
| Installed | NLSolve                       | v4.5.1   |
| Installed | ADTypes                       | v1.19.0  |
| Installed | OrdinaryDiffEqNonlinearSolve  | v1.15.0  |
| Installed | BracketingNonlinearSolve      | v1.6.0   |
| Installed | LinearSolve                   | v3.47.0  |
| Installed | SimpleNonlinearSolve          | v2.9.0   |
| Installed | IfElse                        | v0.1.1   |
| Installed | NonlinearSolveSpectralMethods | v1.6.0   |
| Installed | LineSearch                    | v0.1.4   |
| Installed | ThreadingUtilities            | v0.5.5   |
| Installed | DiffEqNoiseProcess            | v5.24.1  |
| Installed | JumpProcesses                 | v9.19.1  |
| Installed | ManualMemory                  | v0.1.8   |
| Installed | SparseConnectivityTracer      | v1.1.3   |
| Installed | SparseMatrixColonne           | v0.4.22  |

```
In [7]: using DifferentialEquations
```

Рис. 3.1: Установка и подключение пакета differentialEquations.jl

```
In [4]: # задаём описание модели с начальными условиями:
a = 0.98
f(u,p,t) = a*u
u0 = 1.0
# задаём интервал времени:
tspan = (0.0,1.0)
```

```
Out[4]: (0.0, 1.0)
```

```
In [5]: # решение:
prob = ODEProblem(f,u0,tspan)
sol = solve(prob)
```

```
Out[5]: retcode: Success
Interpolation: 3rd order Hermite
t: 5-element Vector{Float64}:
 0.0
 0.10042494449239292
 0.3521860297865888
 0.6934436122197829
 1.0
u: 5-element Vector{Float64}:
 1.0
 1.1034222047865465
 1.412190871348492
 1.9730384457359196
 2.664456142481427
```

Рис. 3.2: Численное решение

Построение графика, соответствующего полученному решению (рис. 3.3):

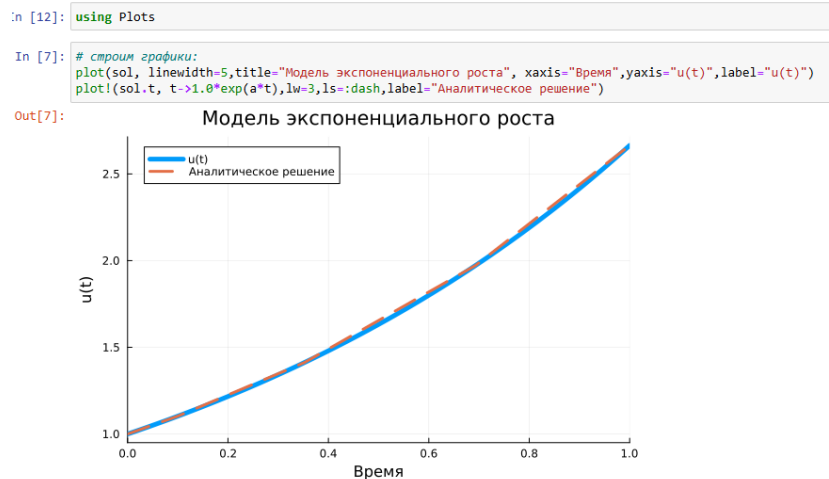


Рис. 3.3: Модель экспоненциального роста

При построении одного из графиков использовался вызов `sol.t`, чтобы захва-

тить массив моментов времени. Массив решений можно получить, воспользовавшись `sol.u`.

Если требуется задать точность решения, то можно воспользоваться параметрами `abstol` (задаёт близость к нулю) и `reltol` (задаёт относительную точность). По умолчанию эти параметры имеют значение `abstol = 1e-6` и `reltol = 1e-3`. Для модели экспоненциального роста (рис. 3.4):

```
In [8]: # задаём точность решения:
sol = solve(prob, abstol=1e-8, reltol=1e-8)
println(sol)
# строим график:
plot(sol, lw=2, color="black", title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label="Численное решение")
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, color="red", label="Аналитическое решение")
```

Рис. 3.4: Задание точности решения

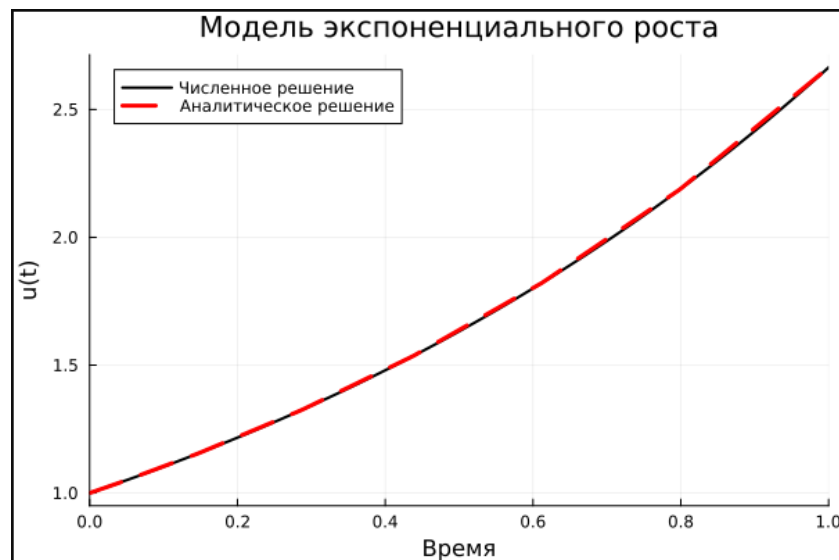


Рис. 3.5: Модель экспоненциального роста

### 3.1.1.2 Система Лоренца

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка:

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = \rho x - y - xz, \\ \dot{z} = xy - \beta z, \end{cases}$$

где  $\sigma$ ,  $\rho$  и  $\beta$  — параметры системы (некоторые положительные числа, обычно указывают  $\sigma = 10$ ,  $\rho = 28$  и  $\beta = 8/3$ ).

Система получена из системы уравнений Навье–Стокса и описывает движение воздушных потоков в плоском слое жидкости постоянной толщины при разложении скорости течения и температуры в двойные ряды Фурье с последующим усечением до первых-вторых гармоник.

Решение системы неустойчиво на аттракторе, что не позволяет применять классические численные методы на больших отрезках времени, требуется использовать высокоточные вычисления.

Численное решение в Julia будет иметь следующий вид (рис. 3.6):

```
In [9]: # задаём описание модели:
function lorenz!(du,u,p,t)
    σ,ρ,β = p
    du[1] = σ*(u[2]-u[1])
    du[2] = u[1]*(ρ-u[3]) - u[2]
    du[3] = u[1]*u[2] - β*u[3]
end
```

Out[9]: lorenz! (generic function with 1 method)

```
In [10]: # задаём начальное условие:
u0 = [1.0,0.0,0.0]
# задаём значения параметров:
p = (10,28,8/3)
# задаём интервал времени:
tspan = (0.0,100.0)
# решение:
prob = ODEProblem(lorenz!,u0,tspan,p)
sol = solve(prob)
```

Out[10]: retcode: Success  
Interpolation: 3rd order Hermite  
t: 1303-element Vector{Float64}:  
0.0  
3.5678604836301404e-5  
0.0003924646531993154  
0.0032623945564751056  
0.009058054066400603  
0.016956441391185063  
0.02768991931434415  
0.04185629724071979  
0.060740340738233165

Рис. 3.6: Численное решение

```
In [11]: # строим график:
plot(sol, vars=(1,2,3), lw=2, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z", legend=false)

Warning: To maintain consistency with solution indexing, keyword argument vars will be removed in a future
e keyword argument idxs instead.
 caller = ip:0x0
 @ Core :-1
```

Out[11]: Аттрактор Лоренца

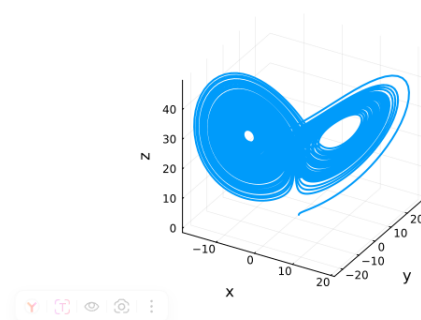


Рис. 3.7: Аттрактор Лоренца

Можно отключить интерполяцию (рис. 3.8):

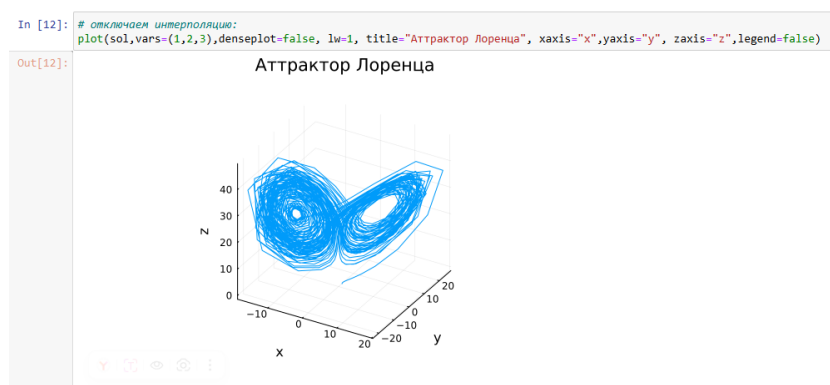


Рис. 3.8: Аттрактор Лоренца (интерполяция отключена)

### 3.1.2 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух видов типа «хищник – жертва»:

$$\begin{cases} \dot{x} = (\alpha - \beta y)x, \\ \dot{y} = (-\gamma + \delta x)y, \end{cases}$$

где  $x$  — количество жертв,  $y$  — количество хищников,  $t$  — время,  $\alpha, \beta, \gamma, \delta$  — коэффициенты, отражающие взаимодействия между видами (в данном случае  $\alpha$  — коэффициент рождаемости жертв,  $\gamma$  — коэффициент убыли хищников,  $\beta$  — коэффициент убыли жертв в результате взаимодействия с хищниками,  $\delta$  — коэффициент роста численности хищников).

Численное решение в Julia будет иметь следующий вид (рис. 3.9):

```
In [13]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("ParameterizedFunctions")

Resolving package versions...
Installed Unitful _____ v1.25.1
Installed MultivariatePolynomials — v0.5.13
Installed Tricks _____ v0.1.13
Installed MutableArithmetics _____ v1.6.7
Installed Bijections _____ v0.2.2
Installed FindFirstFunctions _____ v1.4.2
Installed DomainSets _____ v0.7.16
Installed JuliaFormatter _____ v2.2.0
Installed CompositeTypes _____ v0.1.4
Installed JuliaSyntax _____ v0.4.10
Installed Glob _____ v1.3.1
Installed DispatchDoctor _____ v0.4.26
Installed DynamicPolynomials _____ v0.6.4
Installed Symbolics _____ v6.57.0
Installed MLStyle _____ v0.4.17
Installed CommonMark _____ v0.9.1
Installed ModelingToolkit _____ v10.29.0
Installed ParameterizedFunctions — v5.19.0

In [6]: using ParameterizedFunctions
```

Рис. 3.9: Подключение пакета ParameterizedFunctions

```
In [15]: # задаем описание модели:
lvi = @ode_def LotkaVolterra begin
    dx = a*x - b*x*y
    dy = -c*y + d*x*y
end a b c d

Warning: Independent variable t should be defined with @independent variables t.
@ ModelingToolkit C:\Users\user\.julia\packages\ModelingToolkit\KTR1R\src\utils.jl:121

Out[15]: LotkaVolterra{var"###ParameterizedDiffEqFunction#282", var"###ParameterizedTGradFunction#283", var"###ParameterizedJacobianFunction#284", Nothing, Nothing, System}(Main.var"###ParameterizedDiffEqFunction#282", LinearAlgebra.UniformScaling{Bool}(true), nothing, Main.var"###ParameterizedTGradFunction#283", Main.var"###ParameterizedJacobianFunction#284", nothing, nothing, nothing, nothing, nothing, nothing, [:x, :y], :t, nothing, Model ##Parameterized#281:
Equations (2):
  2 standard: see equations(##Parameterized#281)
Unknowns (2): see unknowns(##Parameterized#281)
X(t)
y(t)
Parameters (4): see parameters(##Parameterized#281)
a
b
c
d, nothing, nothing)

In [16]: # задаем начальное условие:
u0 = [1.0, 1.0]
# задаем значения параметров:
p = (1.5, 1.0, 3.0, 1.0)
# задаем интервал времени:
tspan = (0.0, 10.0)

Out[16]: (0.0, 10.0)
```

Активация Windows  
Чтобы активировать

Рис. 3.10: Численное решение

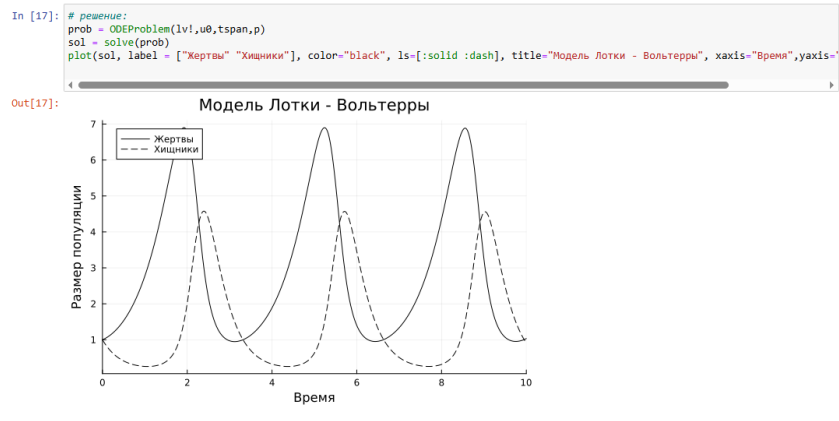


Рис. 3.11: Модель Лотки–Вольтерры: динамика изменения численности популяций

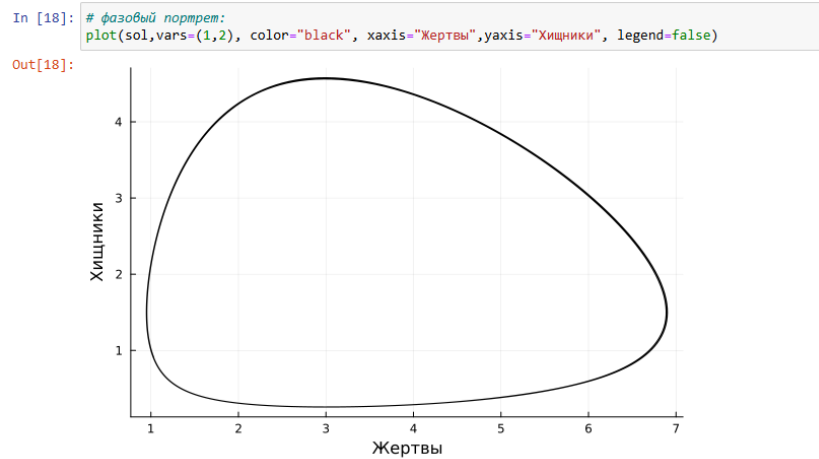


Рис. 3.12: Модель Лотки–Вольтерры: фазовый портрет

## 3.2 Задания для самостоятельного выполнения

1. Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса):

$$\dot{x} = ax, \quad a = b - c,$$

где  $x(t)$  — численность изолированной популяции в момент времени  $t$ ,  $a$  — коэффициент роста популяции,  $b$  — коэффициент рождаемости,  $c$  — коэффициент



смертности. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

```
In [20]: # 1. Модель Мальтуса (экспоненциальный рост)
function malthus_model!(du, u, p, t)
    x = u[1]
    a = p[1]
    du[1] = a * x
end

Out[20]: malthus_model! (generic function with 1 method)

In [21]: # Параметры для модели Мальтуса
a = 0.1 # коэффициент роста (рождаемость 0.15, смертность 0.05)
u0_malthus = [10.0] # начальная численность популяции
tspan_malthus = (0.0, 50.0)
p_malthus = [a]

# Решение модели Мальтуса
prob_malthus = ODEProblem(malthus_model!, u0_malthus, tspan_malthus, p_malthus)
sol_malthus = solve(prob_malthus, Tsit5())

Out[21]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 12-element Vector{Float64}:
 0.0
 0.15849248890571244
 1.562544352940601
 4.329216246946622
 7.810118689783625
12.091558509145196
17.12353945146574
22.893927633296553
29.346068509028527
36.42928760324635
44.0820095407701
50.0
u: 12-element Vector{Vector{Float64}}:
 [10.0]
 [10.159755144250859]
 [11.691236316816065]
 [15.417553734758194]
 [21.83680657762813]
 [33.506542453892735]
 [55.4198698565943]
 [98.68918352511201]
 [188.13989577497495]
 [382.0306480726474]
 [821.1956659558941]
 [1484.0920812236227]

In [22]: # График для модели Мальтуса
p1 = plot(sol_malthus, label="численность популяции",
          xlabel="Время", ylabel="численность",
          title="Модель Мальтуса: Экспоненциальный рост",
          linewidth=2, legend=:topleft)
annotate!(p1, 30, 100, text("a = $a", 10))
```

Рис. 3.13: Модель Мальтуса

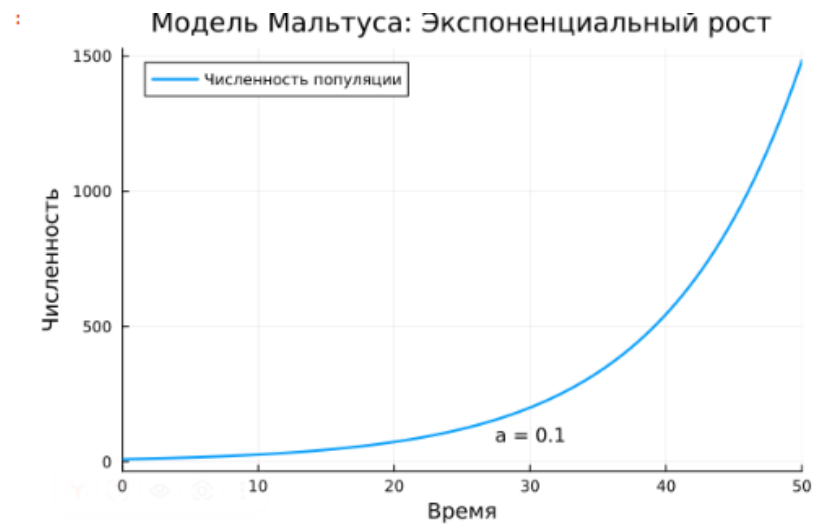


Рис. 3.14: График модели Мальтуса

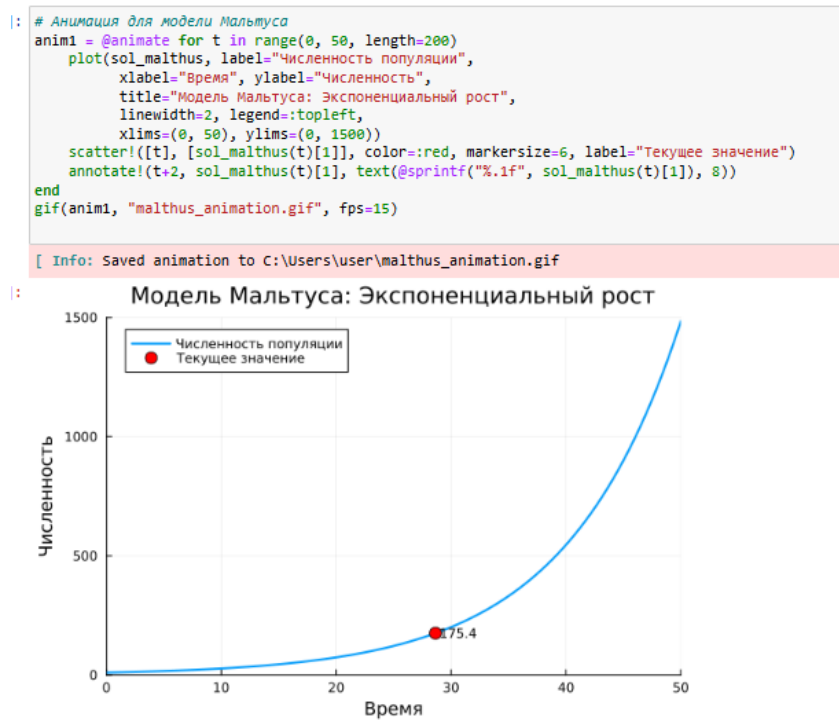


Рис. 3.15: Анимация модели Мальтуса

2. Реализовать и проанализировать логистическую модель роста популяции, заданную уравнением:

$$\dot{x} = rx \left(1 - \frac{x}{k}\right), \quad r > 0, \quad k > 0,$$

$r$  — коэффициент роста популяции,  $k$  — потенциальная ёмкость экологической системы (предельное значение численности популяции). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

```
In [24]: # 2. Логистическая модель
function logistic_model!(du, u, p, t)
    x = u[1]
    r, k = p
    du[1] = r * x * (1 - x/k)
end

Out[24]: logistic_model! (generic function with 1 method)
```

---

```
In [25]: # Параметры для логистической модели
r = 0.2 # коэффициент роста
k = 500.0 # ёмкость среды
u0_logistic = [10.0] # начальная численность популяции
tspan_logistic = (0.0, 100.0)
p_logistic = [r, k]

# Решение логистической модели
prob_logistic = ODEProblem(logistic_model!, u0_logistic, tspan_logistic, p_logistic)
sol_logistic = solve(prob_logistic, Tsit5())

Out[25]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 20-element Vector{Float64}:
 0.0
 0.1385343499475166
 0.9861418836884212
 2.4862334824599643
 4.317341221889021
 6.602358587016193
 9.294648327790243
12.447155676651724
16.098009021227316
20.457695921165335
26.06310153773248
31.085844027481574
37.00038853544241
42.50208008863126
49.30277656446027
56.74524741129299
66.04020150161459
77.26303384728047
91.9540259780935
100.0
u: 20-element Vector{Vector{Float64}}:
 [10.0]
 [10.275169280997861]
 [12.127341185667547]
 [16.232742072928964]
 [23.08066633443723]
 [35.502648601227705]
 [57.896334343415276]
 [98.71875904961598]
 [169.00058022664558]
 [274.8832334489078]
```

Рис. 3.16: Логистическая модель роста популяции

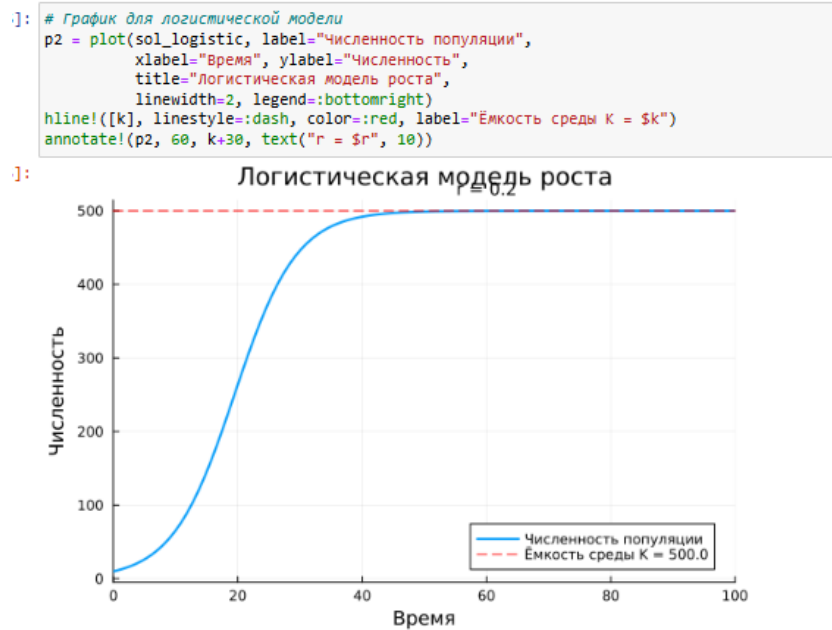


Рис. 3.17: График логистической модели роста популяции

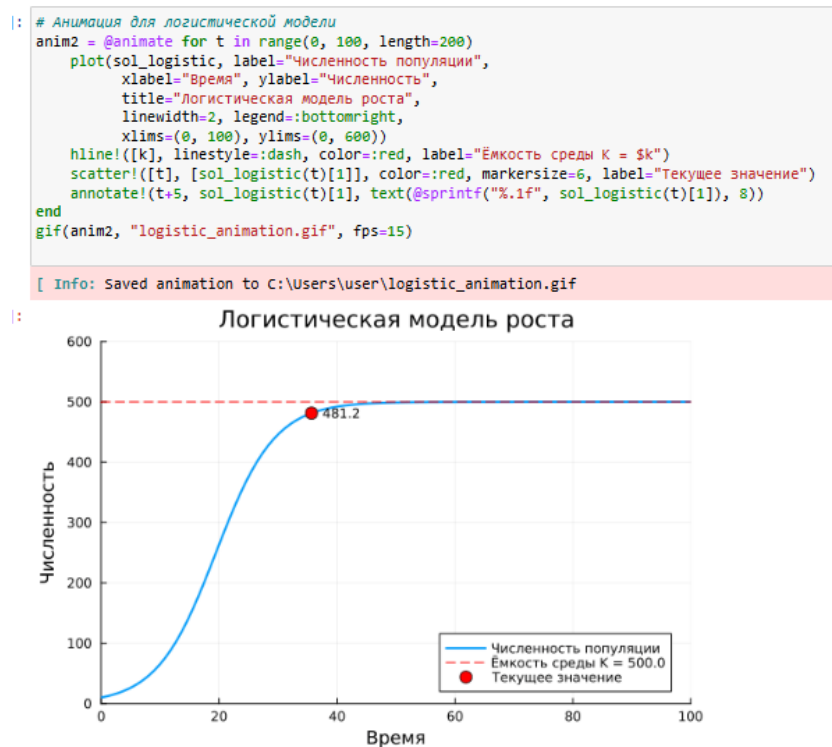


Рис. 3.18: Анимация графика логистической модели роста популяции

### 3. Реализовать и проанализировать модель эпидемии Кермака–Маккендрика

(SIR- модель):

$$\dot{s} = -\beta is,$$

$$\dot{i} = \beta is - \nu i,$$

$$\dot{r} = \nu i$$

где  $s(t)$  — численность восприимчивых к болезни индивидов в момент времени  $t$ ,  $i(t)$  — численность инфицированных индивидов в момент времени  $t$ ,  $r(t)$  — численность переболевших индивидов в момент времени  $t$ ,  $\beta$  — коэффициент интенсивности контактов индивидов с последующим инфицированием,  $\nu$  — коэффициент интенсивности выздоровления инфицированных индивидов. Численность популяции считается постоянной, т.е.  $\dot{s} + \dot{i} + \dot{r} = 0$ . Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

```

i]: # 3. Модель эпидемии SIR
function sir_model!(du, u, p, t)
    s, i, r = u
    β, v = p

    du[1] = -β * i * s # ds/dt
    du[2] = β * i * s - v * i # di/dt
    du[3] = v * i # dr/dt
end

i]: sir_model! (generic function with 1 method)

j]: # Параметры для SIR модели
β = 0.0003 # коэффициент заражения
v = 0.1 # коэффициент выздоровления
u0_sir = [990.0, 10.0, 0.0] # [восприимчивые, инфицированные, переболевшие]
tspan_sir = (0.0, 200.0)
p_sir = [β, v]

# Решение SIR модели
prob_sir = ODEProblem(sir_model!, u0_sir, tspan_sir, p_sir)
sol_sir = solve(prob_sir, Tsit5())

j]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 33-element Vector{Float64}:
 0.0
 0.0014141421188071605
 0.015555563306878765
 0.1569697751875948
 0.7273501964876006
 1.810816243524848
 3.3104998683571587
 5.233807572286311
 7.632226944539838
10.541185813363883
14.051001968636427
18.235777688738175
23.355755602185027

```

Рис. 3.19: Модель эпидемии SIR

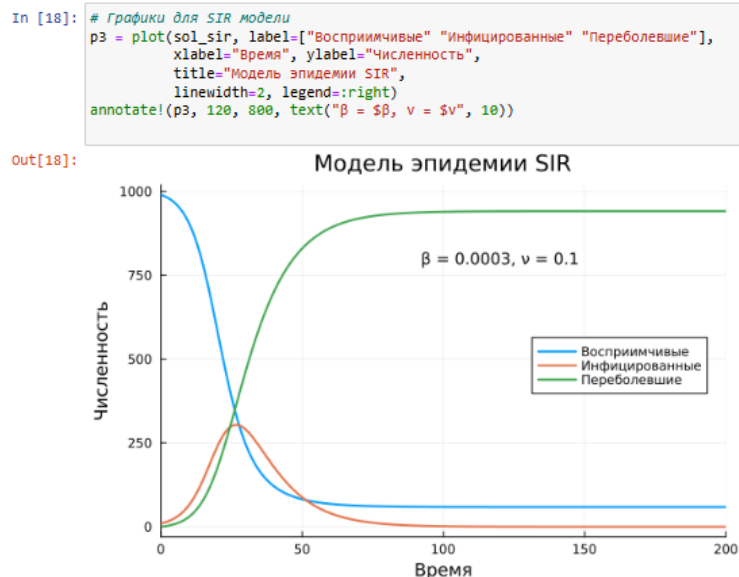


Рис. 3.20: График модели эпидемии SIR

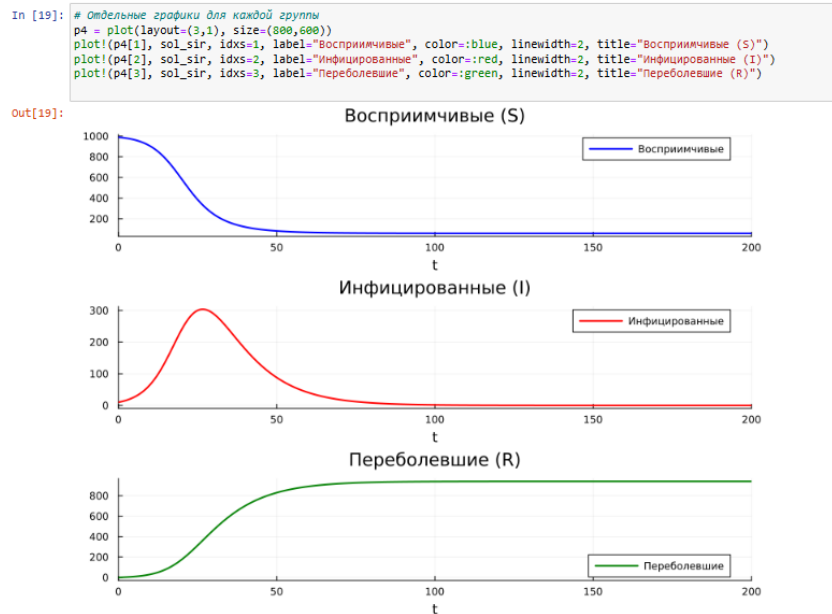


Рис. 3.21: Отдельные графики модели эпидемии SIR

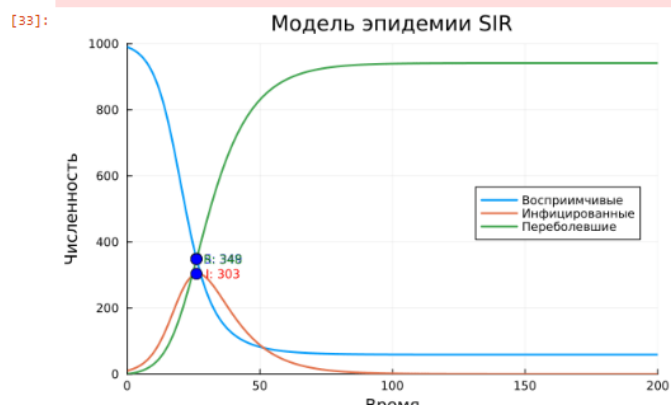
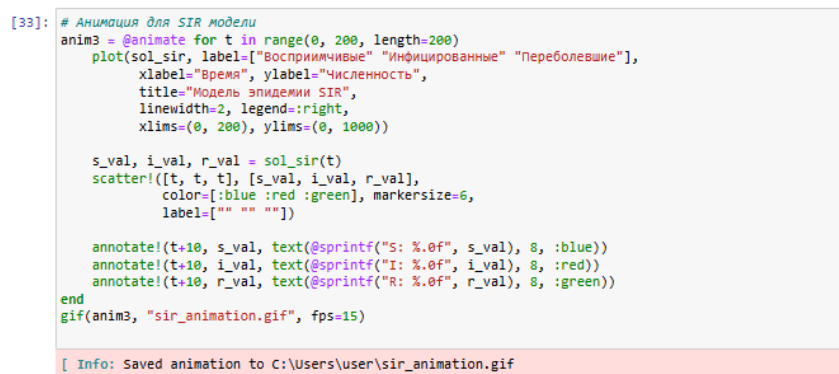


Рис. 3.22: Анимация модели эпидемии SIR

4. Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed):

$$\begin{aligned}\dot{s}(t) &= -\frac{\beta}{N}s(t)i(t), \\ \dot{e}(t) &= \frac{\beta}{N}s(t)i(t) - \delta e(t), \\ \dot{i}(t) &= \delta e(t) - \gamma i(t), \\ \dot{r}(t) &= \gamma i(t).\end{aligned}$$

Размер популяции сохраняется:

$$s(t) + e(t) + i(t) + r(t) = N$$

Исследуйте, сравните с SIR.



```

In [1]: using LinearAlgebra

In [8]: # 4. Модель SEIR
function seir_model!(du, u, p, t)
    s, e, i, r = u
    β, δ, γ, N = p

    du[1] = -β/N * s * i          # ds/dt
    du[2] = β/N * s * i - δ * e   # de/dt
    du[3] = δ * e - γ * i         # di/dt
    du[4] = γ * i                 # dr/dt
end

Out[8]: seir_model! (generic function with 1 method)

In [9]: # Параметры для SEIR модели
β_seir = 0.4 # коэффициент заражения
δ = 0.2 # скорость перехода из экспонированных в инфицированные (1/инкубационный период)
γ = 0.1 # скорость выздоровления
N = 1000 # общая численность популяции
u0_seir = [990.0, 5.0, 5.0, 0.0] # [S, E, I, R]
tspan_seir = (0.0, 200.0)
p_seir = [β_seir, δ, γ, N]

# Решение SEIR модели
prob_seir = ODEProblem(seir_model!, u0_seir, tspan_seir, p_seir)
sol_seir = solve(prob_seir, Tsitsis())

Out[9]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 39-element Vector{Float64}:
 0.0
 0.0034636383408691634
 0.03810002174956079
 0.3240402308575046
 1.0429102531932037
 2.1449881960425676
 3.546109266896558
 5.408499196047138
 7.756465947281605
10.725693127037474
14.266135880642983
18.244126079226845
22.511394425638898
⋮
107.7468492923399
114.98622735933296
122.6176520590499
130.64468964308764
139.0182631069364
147.6904495203093
156.61540551835887
⋮

```

Рис. 3.23: Модель SEIR

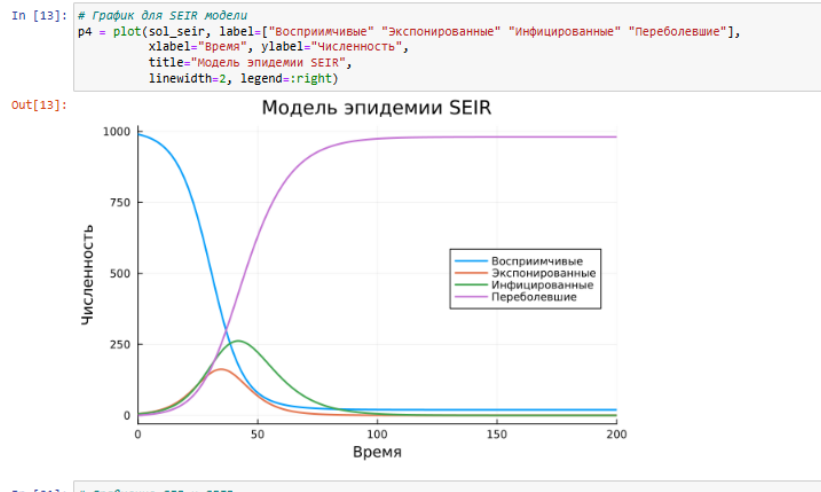


Рис. 3.24: График модели SEIR

```

In [21]: # Сравнение SIR и SEIR
# Пересчитаем SIR с аналогичными параметрами для сравнения
 $\beta_{sir} = 0.4$ 
 $v = 0.1$ 
 $u0\_sir\_comp = [990.0, 10.0, 0.0]$ 
 $p\_sir\_comp = [\beta_{sir}, v]$ 

prob_sir_comp = ODEProblem(sir_model!, u0_sir_comp, tspan_seir, p_sir_comp)
sol_sir_comp = solve(prob_sir_comp, Tsit5())

Out[21]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 1178-element Vector{Float64}:
 0.0
 0.0013148574399352122
 0.002427928030041821
 0.003945915418994764
 0.005583244431774284
 0.007498676528085306
 0.009663980012335486
 0.012137185432393652
 0.015473371868570496
 0.01807190103094005
 0.02069316927400615
 0.023035509575277665
 0.025422389622347538
 ⋮
101.14482264372444
108.73461180799299
116.56080644492744
124.60962793432192
132.8721031944342
141.39375632215157
150.28240741161383
159.7389167822162
170.04501621820732
181.56073982103294
194.74624326185997
200.0
u: 1178-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [983.1962518557092, 16.80202409565559, 0.0017240486352124208]
 [974.0182891977431, 25.977642087857777, 0.0040607143990833985]
 [953.3341714360763, 46.65639371249186, 0.009434851431785574]
 [913.8963136071146, 86.08368948978011, 0.019996903105216006]
 [831.483404059266, 168.47297283818165, 0.043623102552351356]
 [674.8678857606217, 325.0363179951418, 0.0957962442365044]
 [435.687473225092, 564.1073325074575, 0.20519397003320905]
 [168.99363388604526, 830.5644537267315, 0.4419123864233578]
 [67.14135993127603, 932.1859374123818, 0.6727026563422629]
 [24.633913472765848, 974.4426125786271, 0.92347394860713]
 [9.811139180067094, 989.035170431807, 1.1536903873258573]
 [3.8053221242720054, 994.8041167090706, 1.3905611666573348]

```

Рис. 3.25: Сравнение SIR и SEIR



Рис. 3.26: Графики сравнения SIR и SEIR

```

In [24]: # Анализ SEIR vs SIR
peak_i_seir, t_peak_seir = findmax(sol_seir[3,:])
peak_i_sir, t_peak_sir = findmax(sol_sir_comp[2,:])

Out[24]: (997.6785241468378, 16)

In [25]: println("SIR модель:")
println("Пик инфекции: $(round(peak_i_sir, digits=1)) в момент t=$(round(t_peak_sir, digits=1))")
println("SEIR модель:")
println("Пик инфекции: $(round(peak_i_seir, digits=1)) в момент t=$(round(t_peak_seir, digits=1))")
println("Задержка пика в SEIR: $(round(t_peak_seir - t_peak_sir, digits=1)) единиц времени")

SIR модель:
Пик инфекции: 997.7 в момент t=16.0
SEIR модель:
Пик инфекции: 262.1 в момент t=17.0
Задержка пика в SEIR: 1.0 единиц времени

```

Рис. 3.27: Анализ результатов

5. Для дискретной модели Лотки–Вольтерры:

$$X_1(t+1) = aX_1(t)(1 - X_1(t)) - X_1(t)X_2(t),$$

$$X_2(t+1) = -cX_2(t) + dX_1(t)X_2(t).$$

с начальными данными  $a = 2, c = 1, d = 5$  найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

```

In [26]: # 5. Дискретная модель Лотки-Вольтерры
function lotka_volterra_discrete(X, a, c, d)
    x1, x2 = X
    x1_next = a * x1 * (1 - x1) - x1 * x2
    x2_next = -c * x2 + d * x1 * x2
    return [x1_next, x2_next]
end

Out[26]: lotka_volterra_discrete (generic function with 1 method)

In [27]: # Параметры
a = 2.0
c = 1.0
d = 5.0

Out[27]: 5.0

In [28]: # Аналитическое нахождение точек равновесия
# Решаем систему
eq1 = [0.0, 0.0]

# Точка равновесия 2: находим из системы уравнений
x1_eq = (1 + c) / d
x2_eq = a * (1 - x1_eq) - 1
eq2 = [x1_eq, x2_eq]

Out[28]: 2-element Vector{Float64}:
 0.4
 0.19999999999999996

In [29]: # Численное решение
function simulate_lotka_volterra(T, x0, a, c, d)
    trajectory = zeros(2, T)
    trajectory[:, 1] = x0

    for t in 1:T-1
        trajectory[:, t+1] = lotka_volterra_discrete(trajectory[:, t], a, c, d)
    end

    return trajectory
end

Out[29]: simulate_lotka_volterra (generic function with 1 method)

In [30]: # Симуляция
T = 1000
x0_1 = [0.1, 0.1] # Начальные условия близко к нетривиальной точке равновесия
x0_2 = [0.5, 0.2] # Другие начальные условия

traj1 = simulate_lotka_volterra(T, x0_1, a, c, d)
traj2 = simulate_lotka_volterra(T, x0_2, a, c, d)

Out[30]: 2×1000 Matrix{Float64}:
 0.5  0.4  0.36  0.3528  0.371992  ...  0.4  0.4  0.4  0.4  0.4  0.4  0.4
 0.2  0.3  0.3   0.24   0.18336   ...  0.2  0.2  0.2  0.2  0.2  0.2  0.2

```

Рис. 3.28: Модель Лотки-Вольтерры

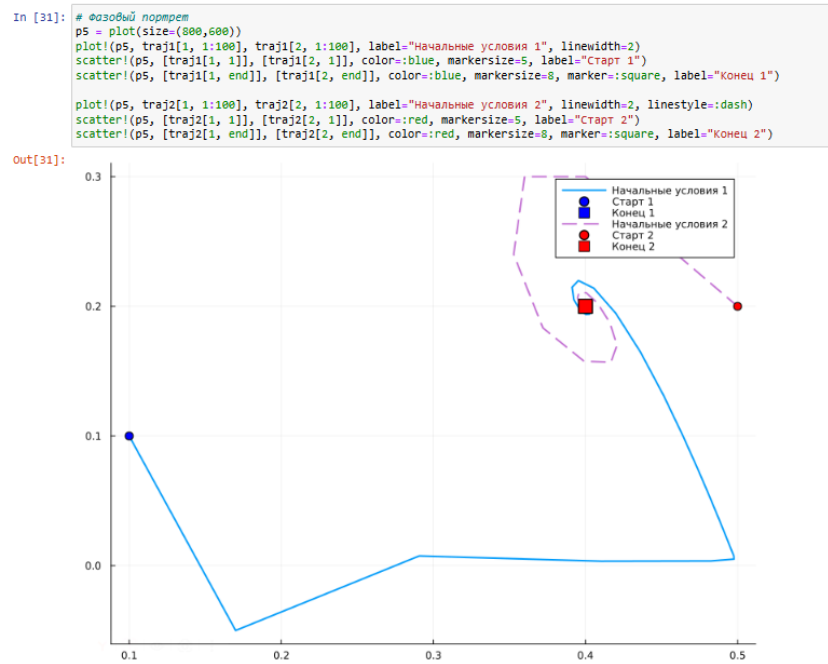


Рис. 3.29: Фазовый портрет модели Лотки-Вольтерры

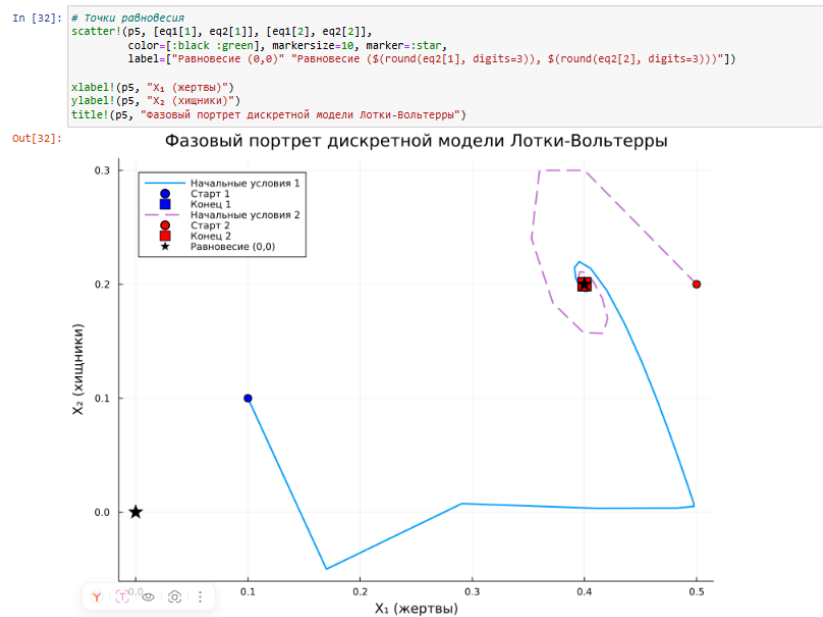


Рис. 3.30: Фазовый портрет с точками равновесия

```

In [33]: # Проверка сходимости к точке равновесия
final_state1 = traj1[:, end]
final_state2 = traj2[:, end]

println("\nПроверка сходимости к точке равновесия:")
println("Траектория 1: конечное состояние = (${round(final_state1[1], digits=4)}, ${round(final_state1[2], digits=4)}))")
println("Траектория 2: конечное состояние = (${round(final_state2[1], digits=4)}, ${round(final_state2[2], digits=4)}))")
println("Точка равновесия: (${round(eq2[1], digits=4)}, ${round(eq2[2], digits=4)}))")

Проверка сходимости к точке равновесия:
Траектория 1: конечное состояние = (0.4, 0.2)
Траектория 2: конечное состояние = (0.4, 0.2)
Точка равновесия: (0.4, 0.2)

```

Рис. 3.31: Проверка сходимости к точке равновесия

6. Реализовать на языке Julia модель отбора на основе конкурентных отношений:

$$\dot{x} = \alpha x - \beta xy,$$

$$\dot{y} = \alpha y - \beta xy.$$

Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```

In [55]: # 6. Модель конкурентных отношений
# Параметры модели
α = 1.0 # скорость роста
β = 0.5 # интенсивность конкуренции
x_0 = 2.0 # начальная численность вида x
y_0 = 1.0 # начальная численность вида y
t_0 = 0.0
t_end = 10.0

Out[55]: 10.0

In [56]: # Определение системы ОДУ
function competition!(du, u, p, t)
    x, y = u
    α, β = p
    du[1] = α * x - β * x * y # dx/dt
    du[2] = α * y - β * x * y # dy/dt
end

# Начальные условия и параметры
u0 = [x_0, y_0]
p = [α, β]
tspan = (t_0, t_end)

# Решение задачи
prob = ODEProblem(competition!, u0, tspan, p)
sol = solve(prob, Tsit5(), dt=0.01)

Out[56]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 3153-element Vector{Float64}:
 0.0
 0.01
 0.10999999999999999
 0.37678003409937727
 0.7232752208766946
 1.1138834207858617
 1.5433375211743245
 1.9892782626688204
 2.326462908978563
 2.55785117922192
 2.7381644115150587
 2.887630556977937
 3.027713983406258
 ⋮
 9.996800450039963
 9.997119853821104
 9.997439155614666
 9.997758355482373
 9.998077453510213
 9.998396449735521
 9.998715344244195
 9.999034137073508
 9.999352828309267
 9.999671417988683
 9.9998990619747
10.0
u: 3153-element Vector{Vector{Float64}}:
 [2.0, 1.0]
 [2.0100251255742214, 0.9999749584900531]
 [2.11319997730996, 0.9969219068276245]

```

Рис. 3.32: Модель конкурентных отношений





```

In [47]: # 7
# Параметры
w_0 = 2π * 1.0 # циклическая частота (например, 1 Гц)
x_0 = 1.0      # начальное положение
y_0 = 0.0      # начальная скорость
t_0 = 0.0      # начальное время
t_end = 5.0    # конечное время

# Определение системы ОДУ
function harmonic_oscillator!(du, u, p, t)
    du[1] = u[2]
    du[2] = -p[1]^2 * u[1]
end

# Начальные условия
u0 = [x_0, y_0]
p = [w_0]
tspan = (t_0, t_end)

```

```

Out[47]: (0.0, 5.0)

```

```

In [48]: prob = ODEProblem(harmonic_oscillator!, u0, tspan, p)
sol = solve(prob, Tsit5(), dt=0.01)

```

```

Out[48]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 45-element Vector{Float64}:
 0.0
 0.01
 0.03400363638899946
 0.07100415632080664
 0.11796565172557918
 0.1784133164466131
 0.24937851251142662
 0.32504895258497274
 0.4094533238024084
 0.5076610056338231
 0.606705726541759
 0.7130789990912356
 0.8289705957472839
 ⋮
 3.551799863127357
 3.6835282345853866
 3.823190731151501
 3.950226629183977
 4.0923564035142705
 4.220162018389925
 4.362395860950445
 4.492284398096584
 4.633586098554436
 4.766009210715533
 4.904969336863321
 5.0
u: 45-element Vector{Vector{Float64}}:
 [1.0, 0.0]
 [0.9980267284255999, -0.39452446974157385]
 [0.9772632788118577, -1.3322202552590783]

```

Рис. 3.35: Консервативный гармонический осциллятор

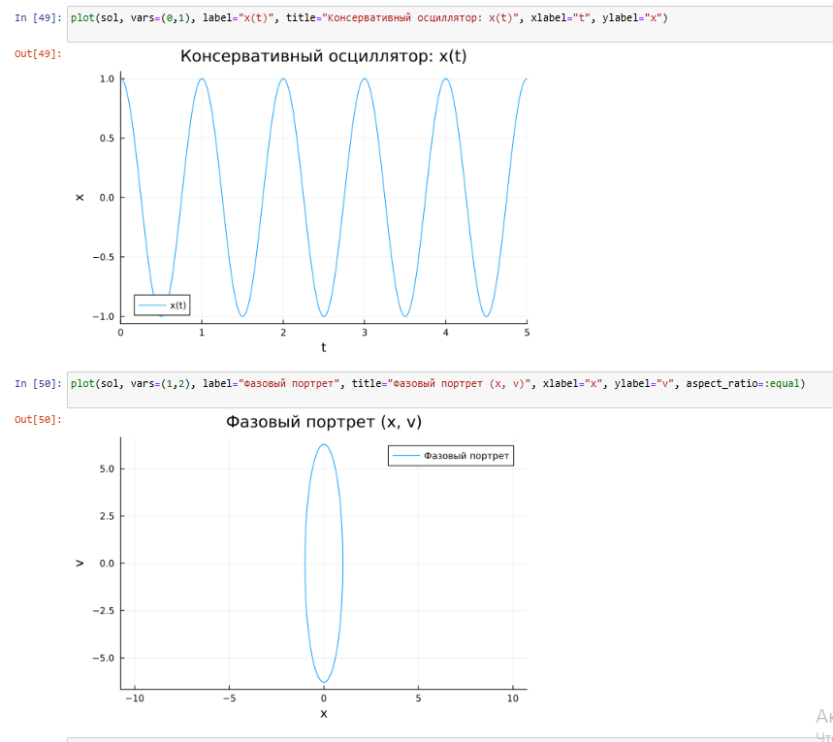


Рис. 3.36: График консервативного гармонического осциллятора и фазовый портрет

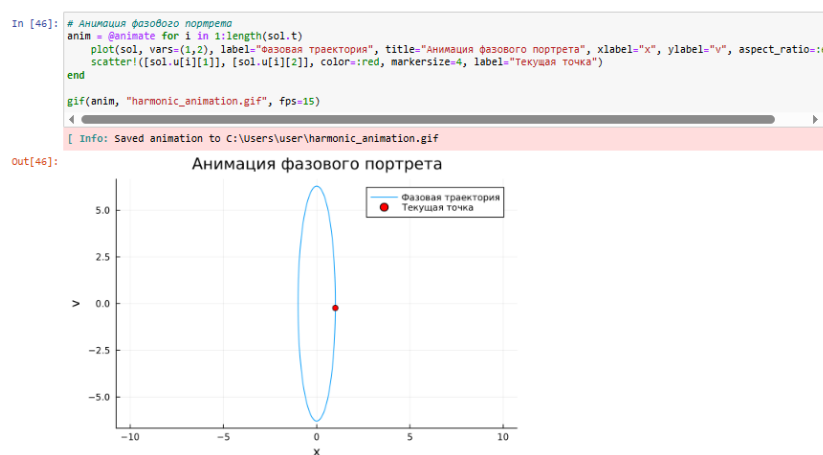


Рис. 3.37: Анимация фазового портрета

8. Реализовать на языке Julia модель свободных колебаний гармонического осциллятора:

$$\ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0, \quad x(t_0) = x_0, \quad \dot{x}(t_0) = y_0,$$

где  $\omega_0$  — циклическая частота,  $\gamma$  — параметр, характеризующий потери энергии. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
In [51]: # Параметры
ω_0 = 2π * 1.0 # циклическая частота
γ = 0.3 # коэффициент затухания (подбираем для слабого затухания)
x_0 = 1.0 # начальное положение
y_0 = 0.0 # начальная скорость
t_0 = 0.0
t_end = 10.0

# Определение системы ОДУ
function damped_oscillator!(du, u, p, t)
    du[1] = u[2]
    du[2] = -2 * p[1] * u[2] - p[2]^2 * u[1]
end

# Начальные условия
u0 = [x_0, y_0]
p = [γ, ω_0]
tspan = (t_0, t_end)

# Создание задачи и её решение
prob = ODEProblem(damped_oscillator!, u0, tspan, p)
sol = solve(prob, Tsit5(), dt=0.01)

Out[51]: retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 83-element Vector{Float64}:
 0.0
 0.01
 0.03406395379569218
 0.07119871188563381
 0.11843317655556705
 0.17929488405418362
 0.2502288304601078
 0.3260651559174538
 0.4089860777862761
 0.5068697177322483
 0.6039213675070406
 0.7094542900812819
 0.8243779513440989
 ⋮
 8.552492053399936
 8.682670086529862
 8.820294927750208
 8.947853854563938
 9.086879584427475
 9.210192515908595
 9.351832472349045
 9.475339241912419
 9.61608962344573
 9.741922240619147
 9.882577899161241
10.0
u: 83-element Vector{Vector{Float64}}:
 [1.0, 0.0]
 [0.9980306687965619, -0.3933432600830469]
 [0.9773373916105171, -1.3710017764766986]
```

Рис. 3.38: Свободные колебания гармонического осциллятора

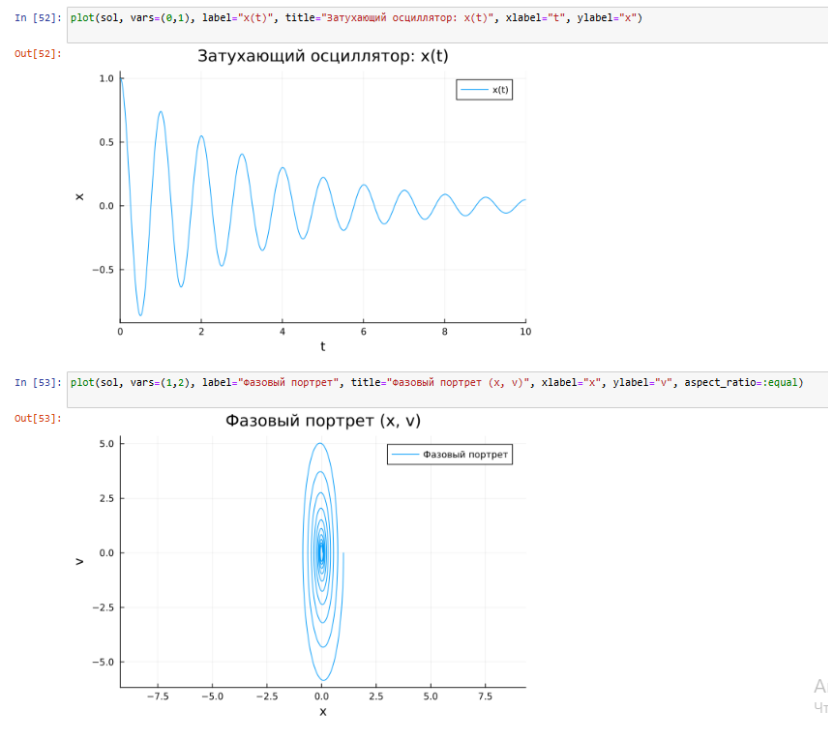


Рис. 3.39: График затухающего гармонического осциллятора и его фазовый портрет

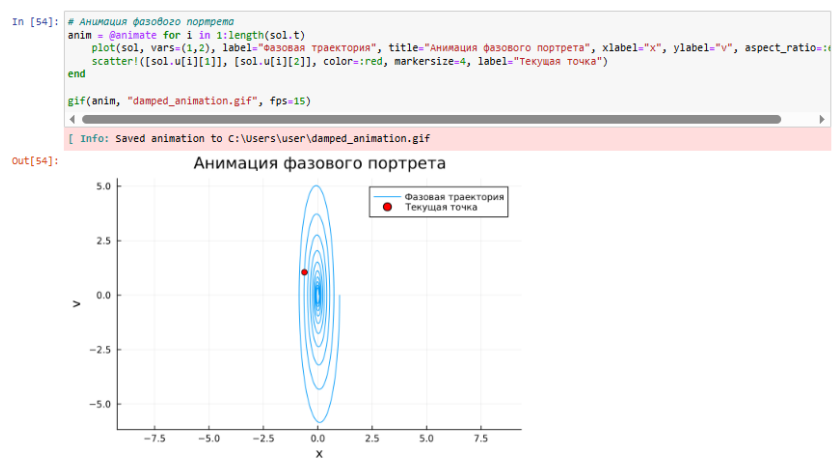


Рис. 3.40: Анимация фазового портрета

## **4 Выводы**

В результате выполнения данной лабораторной работы я освоила специализированные пакеты для решения задач в непрерывном и дискретном времени.